

VoxAI — Agent IA d'Analyse de Sentiment des Avis Clients

PFF FQIA — Sujet N°5 (2026)

Agence de marketing digital — Prototype d'application intégrant IA et automatisation pour l'analyse automatique du sentiment des avis clients.

1. Contexte et Problématique

1.1 Le Problème

Les agences de marketing digital gèrent des dizaines de clients qui reçoivent chacun des centaines de retours provenant de canaux dispersés :

Canal	Exemple	Volume typique
Réseaux sociaux	Facebook, Twitter, Instagram	~50-200 messages/jour
Messagerie instantanée	WhatsApp, Messenger	~20-80 messages/jour
Emails	Support client, réclamations	~10-50 emails/jour
Plateformes d'avis	Google Reviews, Trustpilot	~5-30 avis/jour

Problèmes concrets rencontrés par les équipes marketing :

- Volume ingérable** — Impossible de lire manuellement 300+ messages par jour.
- Dispersion des sources** — Les avis sont répartis sur 5 à 10 plateformes différentes, aucune vue unifiée.
- Subjectivité humaine** — Deux analystes peuvent classer le même avis différemment.
- Lenteur de réaction** — Le temps qu'un rapport soit compilé manuellement (1-2 jours), une crise de réputation peut déjà exploser.
- Absence de tendances** — Sans outil, il est impossible d'identifier que "le service client" est mentionné négativement 3x plus cette semaine que la précédente.

1.2 La Solution : VoxAI

VoxAI est un **agent IA** qui automatise entièrement cette chaîne :

Étape	Module	Fonctionnalités
Entrée	Sources (WhatsApp, Email, Google, Facebook, Twitter)	Données brutes multi-canaux
1	MODULE 1 — COLLECTE + Association Produit	Importation manuelle, JSON, CSV. Stockage SQLite structuré

2	MODULE 2 — ANALYSE IA (NLP + Classification)	Transformers (BERT) pour Sentiment + Score. spaCy pour Mots-clés + Thèmes
3	MODULE 3 — RAPPORT (Génération automatique)	Graphiques interactifs. Points forts / faibles. Évolution temporelle

Résultat : En quelques secondes, l'agence obtient un tableau de bord en temps réel avec des indicateurs exploitables, au lieu de passer des heures à compiler des données manuellement.

2. Architecture Technique

2.1 Vue d'ensemble

C	Composant	Détail
Utilisateur	Navigateur Web (React)	localhost:5173
	Communication	HTTP via Axios
Backend API	FastAPI (Python)	localhost:8000
	Routes REST API	main.py
	NLP Engine	Transformers + spaCy (nlp_engine.py)
	Schémas de validation	Pydantic
Base de données	SQLite (reviews.db)	
	Table <code>products</code>	id, name, description, created_at

Table `reviews`

id,
source,
text,
sentiment,
score,
confidence,
keywords,
themes,
product_id
(FK),
date

2.2 Pile Technologique

Couche	Technologie	Rôle	Justification
Frontend	React 18 + Vite	Interface utilisateur	SPA rapide, composants réutilisables
Styling	Vanilla CSS (Glassmorphism)	Design premium	Contrôle total, Dark Mode natif
Graphiques	Recharts	Visualisation de données	Léger, intégré à React, responsive
Icônes	Lucide React	Iconographie	Moderne, tree-shakable
HTTP Client	Axios	Communication API	Intercepteurs, gestion d'erreurs
Backend	FastAPI (Python)	API REST	Async natif, docs auto (Swagger)
ORM	SQLAlchemy	Accès base de données	Abstraction SQL, migrations
BDD	SQLite	Stockage persistant	Zéro configuration, fichier unique
NLP - Sentiment	Transformers (HuggingFace)	Analyse de sentiment	Modèle BERT multilingue pré-entraîné
NLP - Linguistique	spaCy (fr_core_news_sm)	Extraction de mots-clés	Analyse syntaxique française
Validation	Pydantic	Schémas de données	Validation automatique des types

2.3 Le Modèle IA en Détail

Analyse de Sentiment — `nlptown/bert-base-multilingual-uncased-sentiment`

Ce modèle est un **BERT** (Bidirectional Encoder Representations from Transformers) fine-tuné sur des avis de produits dans 6 langues (français, anglais, allemand, espagnol, italien, néerlandais).

Comment il fonctionne :

- 1. Le texte est **tokenisé** (découpé en sous-mots) par le tokenizer BERT.
- 2. Les tokens passent à travers **12 couches d'attention** qui capturent le contexte bidirectionnel.
- 3. La dernière couche produit une **classification en 5 classes** (1 star → 5 stars).
- 4. Le score est converti en polarité : $\geq 4 \rightarrow$ Positif, $\leq 2 \rightarrow$ Négatif, 3 → Neutre.
- 5. Le modèle renvoie aussi un **score de confiance** (0 à 1) indiquant sa certitude.

Extraction de Mots-clés — spaCy fr_core_news_sm

Ce modèle linguistique français effectue une **analyse syntaxique** complète :

- 1. **Tokenisation** — Découpage en mots.
- 2. **POS Tagging** — Identification de la nature grammaticale (nom, verbe, adjectif...).
- 3. **Dependency Parsing** — Identification des relations entre les mots.
- 4. **Noun Chunks** — Regroupement des groupes nominaux ("service client", "temps de réponse").

Notre code extrait les **Noun Chunks** (en filtrant les pronoms et déterminants) et les **adjectifs isolés** pour former les mots-clés.

Détection de Thèmes — Correspondance Lexicale

8 thèmes prédéfinis (Service Client, Qualité Produit, Prix & Tarifs, Livraison, Interface & UX, Communication, Résultats & ROI, Créativité) sont détectés par correspondance lexicale avec des listes de mots-clés associés.

3. Structure du Projet

Dossier / Fichier	Description
backend/	Backend Python (FastAPI)
backend/__init__.py	Marqueur de package Python
backend/database.py	Modèles SQLAlchemy (Product, Review)
backend/nlp_engine.py	Moteur IA (Transformers + spaCy)
backend/main.py	API REST (tous les endpoints)
frontend-premium/	Frontend React (Vite)
frontend-premium/src/main.jsx	Point d'entrée React
frontend-premium/src/index.css	Design system CSS complet
frontend-premium/src/App.jsx	App principale (3 pages + navigation)
frontend-premium/src/components/KpiCard.jsx	Carte indicateur clé
frontend-premium/src/components/SentimentChart.jsx	Graphiques Pie + Bar (Recharts)

frontend-premium/src/components/TimelineChart.jsx	Graphique évolution temporelle
frontend-premium/src/components/ReviewCard.jsx	Carte d'avis individuel
frontend-premium/src/components/ReportPanel.jsx	Panneau thèmes récurrents
frontend-premium/src/components/AddReviewModal.jsx	Modal ajout d'avis manuel
frontend-premium/src/components/ImportCsvModal.jsx	Modal import CSV (drag & drop)
seed_data.py	Script de peuplement (données de test)
requirements.txt	Dépendances Python
reviews.db	Base de données SQLite (générée au lancement)
docs/	Documentation du projet

4. Endpoints API — Référence Complète

Module 1 — Collecte

Méthode	Route	Description
POST	/products/	Créer un produit/service
GET	/products/	Lister tous les produits
POST	/reviews/	Ajouter un avis (analyse auto)
POST	/import/	Import en lot (JSON)
POST	/import-csv/	Import depuis fichier CSV

Module 2 — Consultation

Méthode	Route	Description
GET	/reviews/?product_id=&sentiment=&source=	Liste filtrée des avis

Module 3 — Rapport

Méthode	Route	Description
GET	/stats/	KPIs globaux (total, score moyen, répartitions)

GET	/stats/timeline/	Score moyen par jour (évolution)
GET	/stats/report/	Thèmes récurrents + points forts/faibles

Tous les endpoints acceptent un paramètre optionnel `?product_id=X` pour filtrer par produit.

5. Comment Lancer le Projet

Prérequis

- Python 3.11+
- Node.js 18+
- pip

Installation

```
# 1. Installer les dépendances Python pip install -r requirements.txt python -m spacy download fr_core_news_sm # 2. Installer les dépendances Frontend cd frontend-premium npm install
```

Lancement (2 terminaux)

```
# Terminal 1 – Backend python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 # Terminal 2 – Frontend cd frontend-premium npm run dev
```

Peuplement de la base (optionnel)

```
python seed_data.py
```

Accès

- **Dashboard** : <http://localhost:5173>
- **API Swagger** : <http://localhost:8000/docs>